

Занятие 3. Что реально используют после 2023: SSD, delta-family, гибриды и sparse attention

Четыре уровня сравнения архитектур

Современные модели удобно сравнивать не по названиям, а по четырём независимым уровням.

1. State equation — уравнение состояния

Уравнение состояния определяет:

- что хранится между соседними шагами последовательности;
- как старая память преобразуется перед записью новой информации;
- каким образом новая информация добавляется в состояние.

Совпадение уравнения состояния у двух моделей ещё не делает их одной архитектурой.

2. Operator — оператор

Оператор описывает математическое отображение всей входной последовательности в выходную. На этом уровне неважно, каким алгоритмом оператор вычисляется.

Один и тот же оператор может иметь:

- рекуррентную форму;
- матричную параллельную форму;
- блочную форму.

Эквивалентность операторов означает равенство вычисляемой функции, но не равенство архитектур.

3. Evaluation algorithm — алгоритм вычисления

Это конкретный способ вычислить оператор:

- последовательно по одной позиции;
- одной большой матричной операцией;

- блоками;
- специализированным recurrent/state-space kernel.

Разные алгоритмы одного оператора могут иметь совершенно разные требования к памяти, число запусков ядер и реальное время работы.

4. Full architecture — полная архитектура

Полная архитектура включает всё, что окружает основной механизм памяти:

- входные и выходные проекции;
- нормализации;
- короткие свёртки;
- гейты;
- число и устройство голов;
- residual-связи;
- порядок и чередование слоёв;
- MoE;
- способ обучения;
- численную точность;
- конкретные вычислительные ядра.

Из совпадения на уровне operator или state equation нельзя делать вывод о совпадении full architecture.

Вектор стоимости архитектуры

Фраза «эта архитектура дешевле» недостаточна без указания координаты стоимости.

Persistent storage — сколько хранится

Это объём информации, который необходимо сохранять между шагами авторегрессионного декодирования.

Для обычного attention это token-indexed KV-cache: отдельная запись на каждую прошлую позицию.

Для finite-state слоя это состояние фиксированного размера.

Main read — сколько нужно прочитать для следующего токена

Хранение и чтение — разные величины.

Механизм может:

- хранить всё и читать всё;
- хранить всё, но читать только небольшую часть;
- хранить только сжатое состояние и читать его целиком.

Именно поэтому sparse attention и finite-state recurrence уменьшают разные статьи стоимости.

Arithmetic — сколько арифметики выполняется

Число FLOPs не определяет реальную скорость само по себе. Особенно при декодировании модель часто ограничена пропускной способностью памяти, а не производительностью вычислительных блоков.

Hardware fit — насколько вычисление удобно железу

Имеют значение:

- размер и форма матричных операций;
- число kernel launches;
- локальность чтения памяти;
- возможность fusion;
- число промежуточных тензоров;
- точность;
- размер batch;
- доступные специализированные kernels.

Поэтому асимптотически более дешёвый алгоритм не обязан быть быстрее на конкретном GPU.

Конечное линейное состояние

Общая форма finite-state recurrence

Удобная общая форма для рассматриваемых механизмов:

$$S_t = S_{t-1}A_t + U_t$$

Здесь:

- S_t — состояние после позиции t ;
- A_t — transition: преобразование старого состояния;
- U_t — новая запись.

Часто новая запись является внешним произведением:

$$U_t = v_t k_t^\top$$

Различные механизмы памяти прежде всего отличаются устройством A_t .

Простое накопление

$$A_t = I$$

Старое состояние не изменяется, а новая запись просто добавляется.

Такое состояние ничего не забывает. Это позволяет сохранять старые вклады, но при большом количестве записей возникает интерференция.

Scalar decay

$$A_t = \gamma_t I$$

Перед новой записью всё состояние умножается на один коэффициент.

Если $0 < \gamma_t < 1$, старые записи постепенно затухают.

Delta rule

$$A_t = I - \beta_t k_t k_t^\top$$

Переход подавляет ту часть старого состояния, которая связана с текущим ключом k_t , после чего записывается новое значение.

В форме обновления:

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

Идея delta rule — не просто добавить новую запись, а сначала исправить память в направлении текущего ключа.

Развёртка рекуррентности

При соглашении о правом умножении transition развёрнутая форма имеет вид:

$$S_t = S_0 A_1 A_2 \cdots A_t + \sum_{i=1}^t U_i A_{i+1} A_{i+2} \cdots A_t$$

При пустом начальном состоянии:

$$S_t = \sum_{i=1}^t U_i A_{i+1} A_{i+2} \cdots A_t$$

Главный смысл развёртки: запись, сделанная в момент i , достигает момента t только после прохождения через все transitions между i и t .

Для матричных transitions порядок множителей существенен. Матричное умножение в общем случае некоммукативно.

Ассоциативность при этом сохраняется: произведения можно группировать блоками, но нельзя произвольно переставлять их местами.

Scalar transition и survival factor

Если transition скалярный,

$$A_t = a_t I$$

то состояние принимает форму:

$$S_t = \sum_{i=1}^t \left(\prod_{j=i+1}^t a_j \right) v_i k_i^\top$$

Здесь естественно разделяются две составляющие.

Content interaction

Вклад записи в ответ зависит от сходства текущего query и старого key:

$$q_t^\top k_i$$

Это отвечает на вопрос: «Насколько содержательно текущий запрос связан с записью i ?»

Temporal survival

Количество старой записи, сохранившееся до момента t :

$$r_{t,i} = \prod_{j=i+1}^t a_j$$

Это survival factor — множитель выживания.

Он не зависит непосредственно от сходства query и key. Если коэффициенты a_j зависят от входных токенов, survival factor зависит от содержимого промежуточного контекста.

При постоянном a и расстоянии Δ :

$$r(\Delta) = a^\Delta$$

Если $0 < a < 1$, вклад убывает экспоненциально. Жёсткой границы памяти нет: существует постепенное затухание.

При $a = 1$:

$$r(\Delta) = 1$$

Старые записи не затухают вообще. Тогда проблема смещается от забывания к интерференции между большим количеством записей.

Из recurrence получается pairwise operator

Пусть чтение состояния выполняется как:

$$y_t = S_t q_t$$

Тогда вклад значения v_i в выход y_t можно записать коэффициентом:

$$\Omega_{ti} = \mathbf{1}[i \leq t] \left(\prod_{j=i+1}^t a_j \right) q_t^\top k_i$$

Выход:

$$y_t = \sum_{i=1}^T \Omega_{ti} v_i$$

или для всей последовательности:

$$Y = \Omega V$$

Матрица Ω причинная: элементы над диагональю равны нулю.

Каждый её элемент содержит три фактора:

1. causality — можно читать только настоящее и прошлое;
2. temporal survival — сколько записи дожило до текущего момента;
3. content interaction — насколько query соответствует key.

Матрица Ω не обязана физически существовать в алгоритме. Она является математическим описанием оператора.

Рекуррентный алгоритм может хранить только компактное состояние и никогда не материализовывать Ω .

Если q_t , k_t и a_t вычисляются из входных данных, то Ω также зависит от входной последовательности. Фиксированной является структура оператора, а не конкретные коэффициенты.

Structured pairwise matrix и compact recurrence

Рекуррентная форма и структурированная матричная форма могут быть двумя точными представлениями одного оператора.

Это не приближение:

- recurrence описывает вычисление через состояние;
- matrix view описывает все попарные коэффициенты;
- оба представления дают один математический результат.

При декодировании рекуррентная форма естественна: состояние уже накоплено, и требуется только один следующий шаг.

При обучении или prefill матричная или блочная форма может позволить лучше использовать параллелизм.

Из существования матричной формы не следует, что recurrence «больше не нужна».

Ограничения finite state

Fixed-size state не означает полное сохранение прошлого

Если состояние имеет фиксированную размерность, это означает только независимость его размера от длины контекста.

Вся информация о прошлом, способная повлиять на будущее, должна пройти через это ограниченное представление.

Произвольный старый токен больше не существует как отдельный адресуемый объект.

Finite state создаёт bottleneck

Если история становится длиннее, состояние не расширяется вместе с ней.

Поэтому модель должна решать:

- что сохранить;
- что ослабить;
- что переписать;
- какие признаки разных записей можно совместить.

Decay, delta rule и gates — разные политики управления этим bottleneck.

Линейность recurrence не гарантирует быстрый параллельный алгоритм

Сам факт линейности по состоянию недостаточен.

Для scalar transition структура очень простая.

Для произвольного матричного A_t пришлось бы протаскивать сложные матричные произведения через последовательность.

Практические быстрые алгоритмы появляются благодаря дополнительной структуре transition. У delta rule, например, transition представляет собой identity минус низкоранговое изменение, задаваемое текущим ключом и коэффициентом.

Structured State Space Duality — SSD

Разрез последовательности

Разделим последовательность после позиции m .

Для записи $i \leq m$ и query $t > m$ survival factor можно разложить в точке границы:

$$\prod_{j=i+1}^t a_j = \left(\prod_{j=i+1}^m a_j \right) \left(\prod_{j=m+1}^t a_j \right)$$

Левая часть зависит только от позиции записи и границы.

Правая часть зависит только от границы и позиции query.

Transition factors можно включить в queries и keys

Определим подготовленный query справа:

$$q_t^{[m]} = \left(\prod_{j=m+1}^t a_j \right) q_t$$

И подготовленный key слева:

$$k_i^{[m]} = \left(\prod_{j=i+1}^m a_j \right) k_i$$

Тогда коэффициенты взаимодействия через границу снова являются обычными скалярными произведениями подготовленных векторов.

Это превращает взаимодействие между двумя большими половинами последовательности в факторизованную структуру фиксированной ширины.

Cross-boundary block

Соберём все подготовленные queries справа в матрицу $Q_{\text{right}}^{[m]}$, а keys слева — в $K_{\text{left}}^{[m]}$.

Тогда блок pairwise-матрицы через границу:

$$\Omega_{\text{right, left}} = Q_{\text{right}}^{[m]} \left(K_{\text{left}}^{[m]} \right)^{\top}$$

Если ширина key равна d_k , ранг такого блока ограничен:

$$\text{rank}(\Omega_{\text{right, left}}) \leq d_k$$

Размер блока может быть огромным — например, миллионы строк и столбцов, — но ранг взаимодействия через границу не может превысить ширину факторизации.

Ширина интерфейса и размер состояния — не одно и то же

Если состояние является матрицей размера $d_v \times d_k$, число хранимых скаляров равно $d_v d_k$.

При этом ранг cross-boundary взаимодействия ограничивается d_k .

То есть d_k характеризует число независимых направлений взаимодействия вдоль последовательности, а не полное число чисел в состоянии.

Semiseparable structure

Матрицы с ограниченным рангом off-diagonal blocks относятся к semiseparable-структурам.

Причинная pairwise matrix при этом может быть плотной во всём нижнем треугольнике.

Semiseparable не означает, что вся матрица low-rank.

Полная матрица может иметь полный ранг.

Ограничивается ранг именно блоков, которые связывают позиции по разные стороны временной границы.

Причинность сама по себе не создаёт rank bound

Произвольная нижнетреугольная матрица тоже причинна.

Однако её cross-boundary block может иметь полный ранг.

Следовательно, ограничение ранга появляется не из causal mask, а из рекуррентной структуры: многие коэффициенты используют общие transition factors и поэтому факторизуются через конечный интерфейс.

Что именно утверждает SSD

SSD связывает:

`structuredrecurrence` $\longleftrightarrow$ `structured causalmatrix` $\longleftrightarrow$ `alternative evaluationalgorithms`

На уровне state equation

Рассматривается определённый класс structured state-space recurrences.

На уровне operator

Им соответствует структурированное, в частности semiseparable, матричное преобразование.

На уровне evaluation algorithm

Один оператор можно считать несколькими способами:

- recurrence;
- structured matrix algorithm;
- chunkwise/state-space algorithm.

На уровне full architecture

Из SSD не следует идентичность Mamba-2 и Transformer.

Не определяются автоматически:

- projections;
- convolution;
- normalization;
- gates;
- heads;
- residual structure;
- порядок блоков;
- обучение.

Название «Transformers are SSMs» относится к математической дуальности определённых структурированных операторов, а не к утверждению, что любые полные Transformer- и SSM-архитектуры являются одной моделью.

Почему SSD важен для алгоритмов

Cross-boundary low-rank structure объясняет возможность chunkwise-вычисления.

Внутри небольшого блока можно использовать плотную матричную операцию.

Между блоками достаточно переносить конечное состояние.

Дальние взаимодействия при этом не отбрасываются: они и в исходной recurrence проходят через состояние.

Таким образом, chunkwise algorithm использует структуру точного оператора, а не приближает его.

Из одной только конечности состояния такой алгоритм не следует. Произвольная нелинейная recurrence тоже может иметь finite state, но не обязана обладать подходящей факторизируемой pairwise structure.

Ёмкость памяти и управляемость памяти

Это две разные характеристики.

Capacity

Сколько чисел находится в состоянии.

Controllability

Насколько независимо модель может решать судьбу разных частей состояния.

Можно увеличить controllability, вообще не увеличив размер состояния.

Scalar gate и channel-wise gate

Пусть состояние содержит две координаты:

$$S = [10, 1]$$

Один scalar gate $\alpha = 0.5$ одинаково масштабирует обе:

$$\alpha S = [5, 0.5]$$

Относительные пропорции между координатами не изменяются.

При channel-wise gate:

$$g = [0.1, 0.9]$$

Определим:

$$G = \text{diag}(g)$$

Тогда:

$$SG = [1, 0.9]$$

Одну координату можно почти удалить, а другую почти сохранить.

Количество координат состояния при этом не увеличилось.

Delta-family как design axis

Delta-family полезно рассматривать не как список названий моделей, а как ось гранулярности управления памятью.

Fixed scalar forgetting

Один фиксированный коэффициент забывания.

Он одинаков для:

- всех позиций;
- всех координат состояния.

Модель не принимает отдельного решения о забывании на каждом токене.

Data-dependent scalar forgetting

Коэффициент вычисляется из текущего входа.

Модель может решать, насколько сильно забывать на каждом шаге, но это всё ещё одно решение для всего состояния.

Global forgetting + targeted delta update

Появляются две независимые ручки:

- α_t — глобально оставить или ослабить старое состояние;
- β_t — насколько сильно исправить память в направлении текущего key.

Типичная форма:

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

Глобальное забывание и targeted rewrite становятся отдельными механизмами.

При этом erase и write внутри самой targeted correction всё ещё связаны коэффициентом β_t .

Vector/channel-wise control

Управление становится покоординатным.

Разные координаты или каналы состояния могут получать разные коэффициенты.

Это увеличивает число независимых решений о памяти, но не устраняет finite-state bottleneck.

Gated DeltaNet

Gated DeltaNet объединяет:

- глобальное забывание;
- delta-style targeted correction.

Его смысл удобно понимать как разделение двух вопросов:

1. сколько старой памяти вообще сохранить;
2. насколько сильно исправить содержимое, связанное с текущим key.

Это более тонкое управление, чем один decay coefficient, но всё ещё finite state фиксированного размера.

RWKV-7

RWKV-7 следует рассматривать отдельно от ранних поколений RWKV.

В RWKV-7 обновление состояния описывается через generalized delta-style mechanism с векторным управлением.

Важно различать:

- сходство по design axis;
- идентичность state equation;
- идентичность full architecture.

RWKV-7 имеет собственные projections, token mixing и устройство блоков, поэтому близость к delta-family не делает её Gated DeltaNet.

Фраза «RWKV использует delta rule» без номера поколения недостаточно точна.

KDA в Kimi Linear

KDA развивает идею более fine-grained управления finite state.

По сравнению с scalar forgetting модель получает больше независимых решений о том, какие компоненты памяти сохранять или изменять.

KDA находится на той же общей оси granularity, что и gated delta mechanisms, но является конкретным механизмом со своей параметризацией и вычислительным алгоритмом.

Большее количество управляющих степеней свободы не является доказательством лучшего качества: меняются оптимизация, стоимость и требования к kernels.

Mamba-3 и развитие transition

Другая ось развития — не granularity gates, а выразительность самого transition.

Mamba-3 движется в этом направлении:

- более общий класс state dynamics;
- использование более богатой структуры состояния;
- в том числе комплекснозначной динамики;
- отдельные изменения организации inputs/outputs и heads.

Вопрос этой оси: насколько более выразительным можно сделать state transition, не потеряв эффективные алгоритмы вычисления.

Gated DeltaNet-2 и разделение операций памяти

Следующий шаг в delta-направлении — отделять всё больше решений друг от друга.

Вместо одной общей ручки targeted update можно отдельно управлять:

- тем, где стирать старую информацию;
- тем, где и насколько сильно записывать новую;
- координатами key-пространства;
- координатами value-пространства.

Общая тенденция — декомпозиция одной операции памяти на несколько независимо управляемых действий.

Почему появляются гибриды

Finite state хорошо подходит для передачи через длинный контекст сжатой сводки.

Но оно не сохраняет отдельную адресуемую запись для каждого токена.

Некоторые задачи требуют retrieval конкретного старого объекта:

- редкого идентификатора;
- точного числа;
- конкретной строки;
- единственного далёкого факта.

Attention сохраняет token-indexed representation и предоставляет прямой механизм такого retrieval.

Гибрид объединяет оба типа памяти вместо требования, чтобы один механизм одинаково хорошо решал обе задачи.

Две топологии hybrid architecture

Layerwise hybrid

Разные слои используют разные mixers.

Например:

```
state layer
state layer
attention layer
state layer
...
```

State-слои используют fixed-size memory.

Attention-слои сохраняют token-indexed cache.

Растущий KV-cache существует только в attention-слоях.

Parallel hybrid

Внутри одного блока существуют несколько ветвей:

```
attention branch ┌
                  │
state branch ───┴── merge
```

Каждый блок одновременно использует state mechanism и attention.

Если attention branch присутствует в каждом блоке, KV-cache также присутствует в каждом блоке.

Поэтому слово «hybrid» само по себе ничего не говорит о коэффициенте памяти: сначала необходимо определить топологию.

Память hybrid architecture

Persistent context representation можно записать как:

$$M_{\text{persistent}} = M_{\text{state}} + M_{\text{KV}}$$

State-компонента не зависит от длины контекста.

KV-компонента для обычного token-indexed attention растёт вместе с длиной.

Если хотя бы часть слоёв сохраняет обычный растущий KV-cache, суммарная память остаётся зависящей от L .

Hybrid уменьшает коэффициент перед L , но не обязательно устраняет саму зависимость.

KV-cache

Для одинаковых attention-слоёв грубая оценка памяти KV-cache:

$$M_{\text{KV}} = 2LN_{\text{attn}}n_{\text{kv}}d_{\text{head}}b$$

где:

- L — длина контекста;
- N_{attn} — число attention-слоёв;
- n_{kv} — число KV-heads;
- d_{head} — размерность головы;
- b — байт на число;
- множитель **2** соответствует отдельным tensors K и V .

Следовательно:

$$M_{\text{KV}} = O(L)$$

Сокращение числа KV-heads или числа attention-слоёв уменьшает коэффициент, но не меняет эту асимптотику.

Память finite state

Если каждый state-слой содержит h_s состояний размера $d_v \times d_k$, оценка:

$$M_{\text{state}} = N_{\text{state}} h_s d_v d_k b_s$$

Здесь длина контекста L отсутствует:

$$M_{\text{state}} = O(1) \quad \text{по отношению к } L$$

Это фундаментально отличается от любого механизма, который продолжает хранить отдельную запись на каждый токен.

Сжатая token-indexed representation не является finite state

Можно уменьшить размер представления K/V для каждого токена.

Тогда:

- каждая запись становится компактнее;
- число записей остаётся равным L ;
- память продолжает расти с длиной.

Если контекст удвоить, такой cache примерно удвоится.

У finite state отдельной записи на токен нет вообще, поэтому увеличение L не меняет размер persistent state.

Интерференция в finite state

Сжатие всей истории в фиксированную матрицу создаёт интерференцию.

Много записей используют одно и то же состояние.

При чтении по query могут появляться вклады других записей, ключи которых не полностью ортогональны нужному.

Decay уменьшает влияние старых записей.

Delta rule исправляет память в выбранном направлении.

Gates позволяют принимать более тонкие решения о том, что оставить.

Но ни один из этих механизмов не создаёт отдельную адресуемую ячейку для каждого токена.

Qwen3.5 как пример layerwise hybrid

В рассмотренной конфигурации используется повторяющийся паттерн:

```
Gated DeltaNet
Gated DeltaNet
Gated DeltaNet
Attention
```

При 15 повторах получается:

- 45 state-слоёв;
- 15 attention-слоёв;
- 60 слоёв всего.

То есть 75% слоёв используют finite state.

Из этого не следует, что KV-cache уменьшен ровно на 75%.

Размер cache конкретного attention-слоя дополнительно зависит от:

- числа KV-heads;
- размерности головы;
- способа представления K/V ;
- precision;
- конкретного attention mechanism.

Процент слоёв и процент байтов — разные величины.

MoE sparsity также является отдельной осью. Разница между total parameters и active parameters per token не описывает хранение контекста.

Kimi Linear как hybrid

Kimi Linear сочетает:

- KDA в state-слоях;
- MLA-подобное сжатое token-indexed representation в attention-слоях.

Это пример, где обе части hybrid самостоятельно уменьшают разные статьи стоимости:

- KDA убирает зависимость state memory от длины;
- MLA уменьшает размер token-indexed записи, не устраняя её рост с L .

Экспериментальные заявления об ускорении такой системы нельзя переносить на другое железо как свойство архитектуры.

Throughput зависит от:

- GPU;
- memory bandwidth;
- kernels;
- operation fusion;
- batch size;
- context length;
- precision;
- tensor shapes;
- числа heads;
- размера state;
- cache quantization;
- режима prefill/decode;
- framework overhead;
- concurrency;
- выбранного baseline.

Falcon-H1 как parallel hybrid

Parallel hybrid может размещать state- и attention-механизмы внутри одного блока.

В Falcon-H1 attention branch соседствует с Mamba-2-style state branch, после чего результаты объединяются.

В такой топологии недостаточно посчитать число «attention layers»: attention присутствует как ветвь внутри каждого соответствующего блока.

Sparse attention: storage и read разделяются

Sparse attention использует принципиально другую стратегию.

Finite state уменьшает объём хранимой истории, сжимая её.

Sparse attention может сохранить большой token-indexed pool целиком, но уменьшить количество данных, которые основная attention-операция читает для конкретного query.

Поэтому:

- finite state прежде всего меняет storage representation;
- sparse retrieval прежде всего меняет read pattern.

Learned sparse retrieval

История разбивается на блоки последовательных токенов:

$$\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_m$$

Для текущего query лёгкая index branch оценивает релевантность блоков.

Выбираются лучшие блоки:

$$I_t = \text{TopK}_j s(\hat{q}_t, \mathcal{B}_j)$$

Selected set:

$$\mathcal{S}_t = \bigcup_{j \in I_t} \mathcal{B}_j$$

Основная ветвь выполняет attention только по выбранному множеству:

$$y_t = \text{Attention}(\hat{q}_t, K_{\mathcal{S}_t}, V_{\mathcal{S}_t})$$

Следующий query может выбрать совершенно другой набор блоков.

Это возможно именно потому, что старые блоки продолжают существовать как отдельные записи в pool.

Почему sparse retrieval выбирает блоки

Выбирать отдельные произвольные токены математически можно, но такое чтение плохо соответствует устройству GPU.

Случайные мелкие чтения:

- ухудшают локальность;
- создают рваный memory access;
- плохо превращаются в крупные matrix operations.

Contiguous blocks позволяют использовать более регулярное чтение и эффективные kernels.

Размер блока создаёт trade-off.

Малые блоки

Преимущества:

- более точный retrieval;
- меньше лишних токенов вокруг нужной информации.

Недостатки:

- больше блоков;
- дороже index;

- больше мелких операций.

Крупные блоки

Преимущества:

- меньше candidates;
- более крупные и удобные memory operations.

Недостатки:

- вместе с нужными позициями читается больше нерелевантных токенов.

Оптимальный block size является свойством модели, kernels и железа, а не универсальной константой.

Пример различия storage и main read

Пусть хранится 1 000 000 токенов, а размер блока равен 64.

Число блоков:

$$\frac{1\,000\,000}{64} \approx 15\,625$$

Если main attention выбирает 32 блока, число выбранных позиций:

$$32 \cdot 64 = 2048$$

То есть token pool содержит около миллиона позиций, а main attention работает примерно с двумя тысячами.

Из этого нельзя заключать, что вся модель стала примерно в 500 раз дешевле.

Остаются:

- полный KV-pool;
- index representation;
- вычисление scores;
- TopK;
- остальные слои модели;
- чтение весов;

- другие memory transfers.

Уменьшилась конкретная координата стоимости — main attention read.

Index branch тоже имеет стоимость

Sparse retrieval требует дополнительного механизма выбора.

Index branch обычно должна:

- иметь компактное представление прошлых элементов или блоков;
- вычислить scores для текущего query;
- агрегировать scores по блокам;
- выполнить TopK.

Поэтому sparsity основного attention не равна sparsity всей системы.

Корректное сравнение должно учитывать индексатор, а не только основной attention kernel.

Что означает exact sparse attention

После выбора $\mathcal{S}_t$ attention внутри этого множества может вычисляться точно.

Это означает точность относительно выбранного множества:

$$y_t = \text{Attention}(q_t, K_{\mathcal{S}_t}, V_{\mathcal{S}_t})$$

Но в общем случае:

$$\text{Attention}(q_t, K_{\mathcal{S}_t}, V_{\mathcal{S}_t}) \neq \text{Attention}(q_t, K_{\text{all}}, V_{\text{all}})$$

Невыбранные элементы полного attention могли иметь ненулевые веса.

Следовательно, «exact» не означает равенство full attention — оно означает отсутствие дополнительной аппроксимации после retrieval.

Sparse attention и finite state имеют разную семантику памяти

Finite state

Старые токены больше не существуют как отдельные объекты.

Они представлены через агрегированное состояние.

Query может прочитать состояние, но не может непосредственно запросить «тот конкретный старый блок».

Sparse attention

Все сохранённые блоки остаются адресуемыми.

Один query может выбрать один набор блоков, следующий — совершенно другой.

Это особенно полезно для редких далёких фактов, которые встречались только один раз.

Sparse attention и hybrid — ортогональные оси

Hybrid отвечает на вопрос:

какие типы mixer использовать в разных слоях или ветвях?

Sparse attention отвечает на вопрос:

какую часть token-indexed history читать внутри attention mechanism?

Поэтому sparse attention может находиться внутри attention-слоя hybrid architecture.

Их нельзя рассматривать как взаимоисключающие альтернативы.

Три ответа на проблему long context

Механизм	Что хранит	Что читает query	Основной trade-off
Fixed-state recurrence	Fixed-size matrix state	Сжатое состояние	Finite-state bottleneck
Hybrid state + attention	State + часть token-indexed KV	Зависит от слоя или ветви	Растущий cache остаётся, но коэффициент может быть меньше
Sparse attention	Большой token-indexed KV-pool	Selected blocks	Storage остаётся большим, main read уменьшается, появляется retrieval cost

Это не три позиции одной шкалы.

Они воздействуют на разные координаты вектора стоимости и могут сочетаться.

GQA и уменьшение числа KV-heads

В обычном multi-head attention у каждой query-head может быть собственная KV-head.

В Grouped Query Attention несколько query-heads используют общие K/V.

Если n_q — число query-heads, а n_{kv} — число KV-heads, то при прочих равных cache уменьшается примерно в:

$$\frac{n_q}{n_{kv}}$$

раз.

Однако количество token-indexed записей остаётся пропорциональным L .

GQA уменьшает одновременно:

- storage;

- объём полного чтения KV-cache.

Но не меняет $O(L)$ -зависимость.

GQA против sparse attention

Пусть sparse attention хранит полный KV-pool и читает долю f .

Тогда main read sparse:

$$R_{\text{sparse}} = f R_{\text{full}}$$

GQA с n_{kv} KV-heads вместо n_q читает:

$$R_{\text{GQA}} = \frac{n_{\text{kv}}}{n_q} R_{\text{full}}$$

Порог равенства:

$$f^* = \frac{n_{\text{kv}}}{n_q}$$

Если:

$$f < f^*$$

sparse attention читает меньше, но хранит больше, чем GQA.

Механизмы тогда несравнимы по двум координатам: один выигрывает по storage, другой — по read.

Если:

$$f = f^*$$

main read одинаков, но GQA хранит меньше.

Если:

$$f > f^*$$

GQA выигрывает и по storage, и по main read в этой упрощённой двухкоординатной модели.

Это не делает GQA функционально эквивалентным sparse attention: механизмы предоставляют разные способы доступа к истории.

Pareto-сравнение механизмов

Механизм нельзя объявить «дешевле», если один вариант:

- хранит меньше;
- но читает больше,

а другой:

- хранит больше;
- но читает меньше.

Такие варианты находятся в разных точках Pareto frontier.

Выбор зависит от реального bottleneck:

- capacity GPU memory;
- memory bandwidth;
- latency;
- throughput;
- необходимость адресного retrieval.

Семь основных ручек управления long-context cost

1. Сокращение числа KV-heads

GQA/MQA уменьшают число K/V-векторов на токен.

Меняется коэффициент перед L , но cache остаётся token-indexed.

2. Сжатие представления K/V

Каждая запись становится компактнее.

Число записей по-прежнему равно числу токенов.

MLA-подобные механизмы относятся сюда.

3. Sliding window

Attention видит только последние W позиций.

Если реализация действительно удаляет всё, что вышло из окна, storage ограничивается:

$$M_{\text{window}} = O(W)$$

Цена — отсутствие прямого доступа к более далёкому прошлому в этом слое.

4. Редкие global attention layers

Большинство слоёв могут использовать local attention или state mechanism, а несколько слоёв сохраняют global token retrieval.

Растущая часть cache остаётся только в global слоях.

5. Sparse retrieval по всей истории

Большой pool сохраняется, но main attention читает выбранные элементы или блоки.

6. KV-cache quantization

Уменьшается количество байт на элемент.

Число записей не меняется:

$$M_{\text{KV}} \propto bL$$

Следовательно, асимптотика по L сохраняется.

7. Eviction

Часть старых записей физически удаляется.

Sliding window — простой eviction policy, где удаляется всё старше фиксированного возраста.

Более сложное eviction требует решать, какие записи можно безопасно забыть.

Local sparse attention и global sparse retrieval

Sparse attention может означать разные механизмы.

Local sparsity

Читается локальное окно или фиксированный pattern.

Если старые элементы действительно удаляются, можно уменьшить и storage.

Global learned retrieval

История сохраняется целиком, а query выбирает произвольные удалённые блоки.

Здесь уменьшается main read, но storage остаётся растущим.

Поэтому слово sparse без описания pattern недостаточно.

Survival factor как эффективный горизонт памяти

Для постоянного decay a вклад записи через Δ шагов:

$$r(\Delta) = a^\Delta$$

При $0 < a < 1$ формального конечного горизонта нет.

Запись не исчезает мгновенно, а постепенно становится настолько малой, что её вклад теряется на фоне других.

Практический горизонт зависит от:

- a ;
- масштаба значений;
- количества других записей;
- численной точности;
- downstream transformations.

Если $a = 1$, горизонт по decay бесконечен, но возникает другая проблема — накопление интерференции.

Современные механизмы памяти ищут компромисс между этими двумя режимами.

Три формы одного causal linear operator

Один оператор может быть реализован тремя существенно различными способами.

Recurrent form

Для additive state:

$$S_t = S_{t-1} + v_t k_t^\top$$

Чтение:

$$y_t = S_t q_t$$

Плюсы:

- состояние фиксированного размера;
- естественное декодирование по одному токenu;
- нет $T \times T$ pairwise matrix.

Минус при обработке всей последовательности — последовательный цикл на уровне алгоритма.

Parallel matrix form

Для всех queries и keys строится pairwise matrix.

Пусть L_{causal} — нижнетреугольная causal mask.

Тогда:

$$Y = ((QK^{\top}) \odot L_{\text{causal}})V$$

Это тот же оператор, но вычисленный большой matrix multiplication.

Преимущество — отличный parallelism на GPU.

Недостаток — промежуточная pairwise matrix имеет размер $T \times T$.

Её память растёт квадратично:

$$M_{\text{pairwise}} = O(T^2)$$

Chunkwise form

Последовательность делится на blocks размера C .

Внутри блока взаимодействия вычисляются матрично.

Взаимодействия с предыдущими blocks проходят через accumulated state.

Для блока с queries Q_b , keys K_b , values V_b :

$$Y_b = Q_b S_{\text{prev}}^{\top} + ((Q_b K_b^{\top}) \odot L_b) V_b$$

После блока состояние обновляется:

$$S_{\text{next}} = S_{\text{prev}} + V_b^{\top} K_b$$

Первое слагаемое результата отвечает за всё прошлое до текущего блока.

Второе — за точные causal interactions внутри блока.

Chunkwise form не является аппроксимацией recurrent operator.

Почему chunkwise form полезна

Она сочетает:

- крупные matrix operations внутри blocks;

- фиксированное состояние между blocks.

При увеличении block size:

- blocks становится меньше;
- операции становятся крупнее;
- растёт объём pairwise work внутри каждого блока.

При уменьшении block size:

- уменьшается локальная quadratic work;
- появляется больше blocks;
- растёт число запусков операций.

Поэтому оптимальный C определяется не только формулой, но и hardware/software stack.

FLOPs и wall-clock time — разные величины

Recurrent form может выполнять меньше арифметики, но оказаться медленнее плохо оптимизированной parallel form.

Причина — большое число маленьких операций.

GPU наиболее эффективен, когда получает достаточно крупные задачи.

Тысячи последовательных маленьких kernel launches могут стоить дороже дополнительной арифметики внутри большой матрицы.

Поэтому:

меньше FLOPs $\nRightarrow$ меньше latency

Асинхронность GPU и корректный benchmarking

CUDA-вызовы обычно асинхронны относительно CPU.

CPU может вернуть управление сразу после постановки kernel в очередь.

Если измерить время обычным CPU-таймером без синхронизации, получится время enqueue, а не время завершения computation.

Синхронизация

Перед чтением результата CPU-таймера требуется дождаться GPU, например через `torch.cuda.synchronize()`.

Для измерения CUDA kernels удобнее использовать `torch.cuda.Event`, потому что события записываются в тот же CUDA stream и измеряют время выполнения на устройстве.

Warm-up

Первые запуски могут включать разовые расходы:

- инициализацию CUDA;
- выбор алгоритмов;
- compilation;
- создание kernels;
- заполнение caches;
- memory allocation.

Поэтому перед основными измерениями выполняются warm-up iterations.

Повторы

Один запуск не является надёжным benchmark.

Нужно собирать несколько samples.

Удобная центральная оценка — медиана.

Разброс между повторениями показывает стабильность измерения.

Peak memory

Для GPU полезно отдельно измерять пик allocator memory.

В PyTorch для этого применяются `torch.cuda.reset_peak_memory_stats()` и `torch.cuda.max_memory_allocated()` .

Перед и после измеряемой операции требуется учитывать асинхронность.

CPU benchmark

На CPU нет CUDA queue, но результат всё равно зависит от:

- числа threads;
- BLAS backend;
- vectorization;
- memory hierarchy.

Для воспроизводимого сравнения число потоков и software stack должны быть зафиксированы.

Что нельзя переносить из benchmark

Результат вида «механизм X быстрее механизма Y» относится только к зафиксированной конфигурации.

Он может измениться при смене:

- GPU;
- версии PyTorch;
- CUDA;
- kernels;
- tensor dimensions;
- block size;
- batch;
- precision;
- compiler;
- fusion;
- context length.

Асимптотика описывает рост математической стоимости.

Benchmark описывает конкретную реализацию на конкретной машине.

Это разные уровни утверждения.

Как читать новую архитектуру

Шаг 1. Найти state equation

Нужно определить:

- что хранится;
- растёт ли память с L ;
- чем является transition;
- есть ли forgetting;
- есть ли targeted correction;
- scalar или vector/channel-wise управление.

Шаг 2. Найти operator

Нужно понять:

- какие позиции могут взаимодействовать;
- существуют ли отдельные token-indexed записи;
- можно ли представить оператор pairwise matrix;
- какую структуру имеет эта матрица.

Шаг 3. Найти evaluation algorithm

Нужно определить:

- recurrence;
- parallel matrix;
- chunkwise;
- scan;
- специализированные kernels;
- sparse retrieval;
- TopK;

- cache access pattern.

Шаг 4. Найти full architecture

Нужно посмотреть:

- какие mixers используются;
- как они чередуются;
- layerwise или parallel hybrid;
- projections;
- normalization;
- convolution;
- heads;
- MoE;
- residual paths.

Шаг 5. Заполнить cost vector

Для каждого механизма отдельно:

- storage;
- read;
- arithmetic;
- hardware fit.

После этого можно сравнивать архитектуры, не смешивая разные типы стоимости.

Типичные неверные выводы

«Оператор можно записать как structured matrix, значит архитектура эквивалентна Transformer»

Нет. Это утверждение про operator и, возможно, evaluation algorithm.

«Semiseparable matrix является low-rank matrix»

Нет. Ограничен rank определённых off-diagonal cross-boundary blocks. Вся matrix может иметь полный rank.

«Fixed-size state полностью сохраняет прошлое»

Нет. Оно сжимает всю необходимую для будущего информацию в representation фиксированного размера.

«Vector gate увеличивает размер памяти»

Нет. Он увеличивает granularity управления существующими координатами.

«Больше gates автоматически лучше»

Нет. Выразительность, оптимизация, вычислительная стоимость и hardware efficiency меняются одновременно.

«Большинство слоёв state-based, значит KV-cache уменьшился на тот же процент»

Нет. Число слоёв и число байтов cache — разные величины.

«Compressed KV является finite state»

Нет. Пока существует отдельная запись на каждый токен, storage остаётся token-indexed и растёт с L .

«Sparse attention хранит только выбранные blocks»

Не обязательно. В global sparse retrieval весь pool сохраняется, а selected set определяет чтение текущего query.

«Exact sparse attention равно full attention»

Нет. Exact означает точное вычисление внутри selected set.

«Sparse attention — альтернатива hybrid architecture»

Нет. Эти механизмы относятся к разным design axes и могут использоваться одновременно.

«Модель линейная по длине»

Фраза недоопределена.

Нужно уточнять, о чём идёт речь:

- storage;
- FLOPs;
- prefill;
- decode;
- конкретный слой;
- всю модель.

«Авторы получили ускорение x раз, значит моя система ускорится так же»

Нет. Ускорение — измеренная характеристика конкретной реализации и baseline.

Система координат современных long-context architectures

Вместо поиска одной «победившей архитектуры» полезнее рассматривать независимые решения.

Structured state

Прошрое сжимается в finite state.

Плюсы:

- storage не растёт с L ;
- дешёвое autoregressive update.

Цена:

- finite-state bottleneck;
- отсутствие отдельной адресуемой записи на каждый токен.

Hybrid

Разные механизмы получают разные роли.

Плюсы:

- state layers дешёво передают сводку;
- attention layers сохраняют explicit retrieval.

Цена:

- token-indexed cache остаётся в части архитектуры.

Sparse retrieval

История остаётся token-indexed, но main attention читает только selected subset.

Плюсы:

- адресуемое далёкое прошлое;
- сокращение main read.

Цена:

- storage может оставаться большим;
- требуется index;
- retrieval может не выбрать нужный блок.

Современная архитектура может одновременно использовать все три идеи.

Главное различие storage semantics

Для оценки long context полезно сначала определить, что происходит с отдельным старым токеном.

Token-indexed cache

Старый токен продолжает существовать как отдельная запись.

Количество persistent records обычно растёт с L .

Compressed token-indexed cache

Старый токен продолжает существовать как отдельная запись, но меньшего размера.

Рост с L сохраняется.

Evicted/windowed cache

Старые записи физически удаляются.

Storage можно ограничить, но удалённое прошлое больше непосредственно недоступно.

Sparse-retrieval pool

Записи хранятся отдельно, но текущий query читает только часть из них.

Finite state

Отдельные старые записи перестают существовать как адресуемые объекты.

История представлена одной сжатой структурой фиксированного размера.

Что читать дальше

Tri Dao, Albert Gu — *Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality* (2024)

[arXiv:2405.21060](https://arxiv.org/abs/2405.21060)

Читать ради SSD: точного соответствия между structured recurrence, semiseparable matrix view и различными алгоритмами вычисления. Это основная работа для понимания того, почему recurrence и большая pairwise matrix могут быть двумя представлениями одного оператора и как из этой структуры получается Mamba-2.

Songlin Yang et al. — *Parallelizing Linear Transformers with the Delta Rule over Sequence Length* (2024)

[arXiv:2406.06484](https://arxiv.org/abs/2406.06484)

Читать ради delta rule и главной вычислительной проблемы матричного transition: как превратить последовательное обновление памяти с targeted correction в практически параллелизуемый алгоритм.

Songlin Yang et al. — *Gated Delta Networks: Improving Mamba2 with Delta Rule* (2024)

[arXiv:2412.06464](https://arxiv.org/abs/2412.06464)

Читать ради соединения forgetting и delta-style correction. Работа показывает, зачем отделять глобальное управление сохранением старой памяти от targeted rewrite и как это превращается в практически используемый механизм.

Bo Peng et al. — *RWKV-7 “Goose” with Expressive Dynamic State Evolution* (2025)

[arXiv:2503.14456](https://arxiv.org/abs/2503.14456)

Читать ради другой ветви fine-grained state control: generalized delta-style dynamics, векторного управления состоянием и понимания того, почему сходство state update не означает идентичность полной архитектуры.

Kimi Team — *Kimi Linear: An Expressive, Efficient Attention Architecture* (2025)

[arXiv:2510.26692](https://arxiv.org/abs/2510.26692)

Читать ради KDA и реального layerwise hybrid design, в котором state mechanism сочетается со сжатым token-indexed attention. Особенно полезна для разделения трёх понятий: finite state, compressed KV и hybrid topology.

Aakash Lahoti et al. — *Mamba-3: Improved Sequence Modeling using State Space Principles* (2026)

[arXiv:2603.15569](https://arxiv.org/abs/2603.15569)

Читать ради следующего шага по оси выразительности state transition: более богатой динамики состояния и того, как расширять SSM, сохраняя возможность эффективного вычисления.

Technology Innovation Institute — *Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance* (2025)

[arXiv:2507.22448](https://arxiv.org/abs/2507.22448)

Читать ради parallel hybrid topology: state- и attention-механизмы используются как параллельные ветви внутри блоков, а не как разные типы слоёв на разных глубинах.

Jingyang Yuan et al. — *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention* (2025)

[arXiv:2502.11089](#)

Читать ради современного sparse attention и связи между математической разреженностью и реальным исполнением на GPU: block structure, выбор ограниченного множества элементов и необходимость проектировать sparsity с учётом hardware locality.

Tri Dao et al. — *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* (2022)

[arXiv:2205.14135](#)

Читать ради понимания, почему FLOPs недостаточно для оценки скорости. Работа вводит удобный взгляд через memory traffic и IO complexity, который нужен для анализа decode, KV-cache, block algorithms и hardware-efficient attention.